Universidade Leiria • Oeste
Escola de Tecnologia e Gestão

DEPARTAMENTO DE ENGENHARIA INFORMÁTICA
www.dei.estg.ipleiria.pt

Systems Integration

Messaging Systems

Topics:
- Background
- Messaging systems
- Web Services or Messaging?
- Mosquitto Messaging Broker
- Examples and exercises

©2026-2027: {marisa.maximiano, nuno.costa, humberto.ferreira}@ulo.pt

Background

There is no optimal solution that can be applied to all integration problems, which means that, different integration problems require different solutions or approaches (beyond Web Services).

Web Services (studied in previous classes) are part from a set of technologies that support systems or applications integration by direct connection or point-to-point, which means that Web Services are interesting to connect a reduced number of systems, otherwise the integration solution will become too much complex, hard to maintain and without scalability. Take this example:

Suppose a company or institution where 3 heterogeneous applications are used (these applications do not interoperate among themselves). Now, imagine that the company has plans to integrate the 3 applications to get a unified system that mimics the behaviour of a single system.

If Web Services are used, 3 connections are expected to connect all the existing applications, which is not a big deal. However, if the number of applications reach the 10 units, 45 connections must be established and handled, according to the formula **N(N-1)/2**, where N represents the number of applications that must interoperate.

On the one hand, Web Services follow the request/response model and the server, and the client must be active at the same time, and the server must have enough resources to accept a bunch of clients (there is no contention by default). On the other hand, if the server goes offline, the client is responsible for handling all errors that come from that situation and if it is the client that goes down then the response is lost.

The Web Services are very interesting for situations where the client expects an immediate response from the server (**synchronous**).

1. Messaging Systems

Communication based on message passing is a style where one or more queues are used between message sender and message receiver, where both ends can put or retrieve messages from the queues hence following a very flexible and "**disconnected**" way of communication.

A remarkable characteristic of messaging systems is the ability for communication when the ends may not be active at the same time once the messages are stored in the queues (or even persisted in the hard disc) until proper delivery. Besides this, this approach naturally implements contention, avoiding systems saturation in situations of high load. In this way, neither the sender nor the receiver must worry about network communications or offline errors because the messaging system handles that in a transparent way.

Once messaging systems are **asynchronous** by nature (but also support the synchronous approach) they are not very suitable for synchronous requests. Additionally, as messaging systems do not rely in point-to-point connections among applications or systems, in a 3 applications scenario only 3 connections are requested and in a 10 applications scenario only 10 connections are required.

2. Web Services or Messaging?

As already stated before, there is no unique solution for all integration problems. While Web Services are tailored for synchronous interactions or for scenarios with a small number of applications, messaging is tailored for scenarios where asynchronous interactions are required or for scenarios where lots of applications need to interact among themselves (e.g., Facebook, X Social Network/Twitter, etc.). Next, some tips are presented to identify the better approach for systems or applications integration:

- Do you want to handle the network errors or let them to the integration middleware? Messaging systems are asynchronous and can retry operations in the presence of failures.
- Choose a messaging system if clients do not need to actively wait for response.
- Messaging systems are tailored for long running requests once client will not be blocked waiting for the response.
- Messaging systems turn the integration solution high scalable once high request loads are naturally serialized into the processing queues and processing is naturally distributed through the time.
- Web services and messaging systems are not mutually exclusives. It is possible to create a messaging system where the send and receive interface is implemented using Web Services.

Note: Despite messaging systems appearing to solve the problems of Web Service, messaging systems have also problems such as message format or the communication protocol.

Enterprise Service Buses (ESB), which are based on messaging, have additional functionalities such as data transformations, support for multiple communication protocols and even business support logic to address the limitations of the messaging systems. BizTalk is an example of an ESB.

3. Mosquitto Messaging Broker

Eclipse Mosquitto™ [mosquitto.org] is an open source (EPL/EDL licensed) message broker that implements the MQTT protocol [mqtt.org/] versions 3.1.0, 3.1.1 and 5.0 and can be used to connect different systems or applications following the **publish/subscribe model**. Alternatives to Mosquitto can be RabbitMQ, HiveMQ, Kafka, ActiveMQ, etc.

3.1. The Publish/Subscribe model

The publish/subscribe model defines 3 roles: the publisher, the subscriber and the middleman called broker. **Publishers**, as the name says, are the entities that generate and publish events or information, while **subscribers** are the entities that are interested in some type of events or information (they are interested in the work of publishers). By its turn, the **broker** is the entity that mediates communication or connections between publishers

and subscribers. In this perspective, the publishers (producers) don't care about the number of subscribers (consumers) nor the time the subscribers are active.

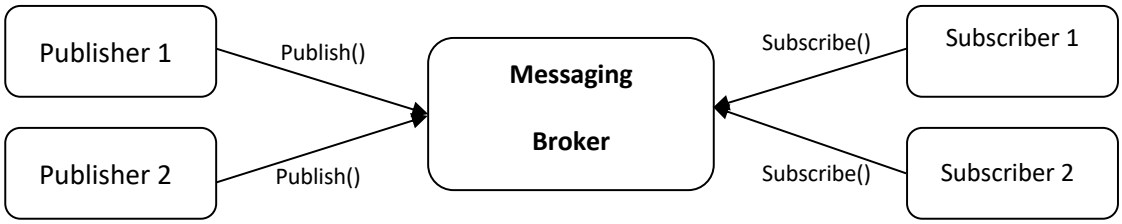


Fig. 1 – Publish/subscribe model

More concretely, the messaging broker creates the **topic** (e.g., channel) abstraction to be possible to handle multiple independent flows of information and to optimize the communication. This allows subscribers to only listen to communication channels (e.g. topics) they are interested to. Apart from permissions and security settings, one publisher can publish in one or more topics and one subscriber can subscribe to one or more topics. Figure 2 presents the publish/subscribe model but now with more detail.

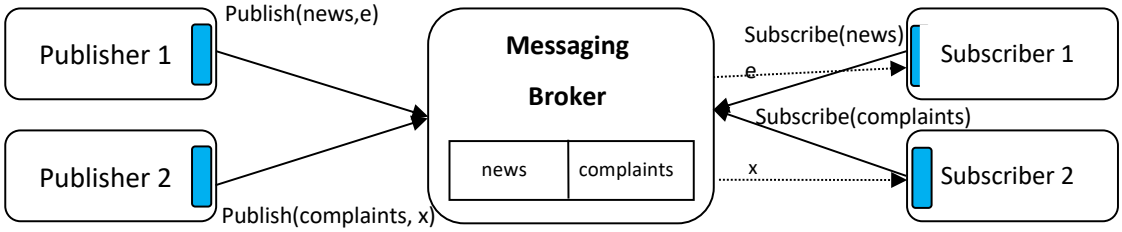


Fig. 2 – Publish/subscribe model with topics and access middleware

The blue rectangles represent the middleware (API) to access the messaging broker, and this middleware plays a very important role in the communications reliability because, usually, that middleware implements the send & forget, plus Store and Forward approaches. This means that, when a publisher calls the **publish()** function it does not care about any kind of errors (e.g. communication) because the send & forget approach is implemented by the local middleware. On the other hand, the store and forward means that all in-transit messages are stored to survive to system downs, system crashes or other kind of system sever problems. In the subscriber's side, the middleware abstracts all the communications and deliver messages (asynchronously) to the subscriber application. Depending on configuration settings, a new subscriber can receive all past messages of a specific topic without any extra line of code.

3.2. The MQTT Protocol

MQTT is a machine-to-machine (M2M)/"Internet of Things" connectivity protocol. It was designed as an extremely lightweight publish/subscribe messaging transport. It is useful for connections with remote locations where a small code footprint is required and/or network bandwidth is at a premium. For example, it has been used in sensors communicating to a broker via satellite link, over occasional dial-up connections with healthcare providers, and in a range of home automation and small device scenarios. It is also ideal for mobile applications because of its small size, low power usage, minimized data packets, and efficient distribution of information to one or many receivers [from mqtt.org]

By default, the MQTT protocol uses the TCP/IP port 1883 (reserved with IANA) and port 8883 for using MQTT over SSL. MQTT is in the process of undergoing standardization at OASIS.

In terms of security, MQTT allows the usage of login and password with the MQTT packet and SSL if the SSL impact is acceptable to the system. Additional security could be added by encrypting data at application layer (not included in the protocol).

3.3. A word about QoS

MQTT defines three **levels of QoS** that could be assigned to each message being published: "at most once", "at least once" and "exactly once". Do not forget that higher QoS levels consume more resources than the lower ones. This could be important if you are dealing with resource poor hardware (e.g., sensor nodes).

3.4. Client libraries (and .NET)

MQTT, in this case Mosquitto, has lots of client APIs that cover the most known programming languages and systems such as Arduino, Bash, C, C++, Delphi, Erlang, Java, Javascript, Node.js, .NET, Objective-C, Perl, PHP, Python, Ruby, Qt, Swift, etc. See https://github.com/mqtt/mqtt.github.io/wiki/libraries for more details.

For this worksheet, we will use the **MQTTnet** library to add MQTT capability to the .Net applications.

To find out if Mosquitto is ready to be used on your local machine, execute the following commands in separated consoles:

```
mosquitto_sub -v -t 'news'                    (to subscribe news topic/channel)
mosquitto_pub -t 'news' -m “Hello World!”     (to post message to topic)
```

To integrate the **MQTTnet** .NET client library into a C# project do the following:

```
dotnet add package MQTTnet
```

In you class file you need to import:

```
using MQTTnet;
```

> **Important note:** eduroam wireless network does not allow P2P connections; hence it is not possible to have the messaging broker in one node and the publishers/subscribers in different nodes. One way to overcome this network limitation is to rely on **labs** network or in **test.mosquitto.org** free instance.

To connect to an online broker, e.g. **test.mosquitto.org** you need to use the **-h** option.

```
mosquitto_sub -h  test.mosquitto.org -t "news"
mosquitto_pub -h  test.mosquitto.org -t "news" -m "Hello"
```

4. Examples and Exercises

For the next exercises, you are free to use either a locally hosted MQTT broker or the public MQTT broker provided at **test.mosquitto.org**. Be aware that connections to the public broker may be blocked while using the **eduroam** wireless network.

4.1. Example 1 – MQTT Publisher and Subscriber

Example 1 demonstrates a single form-based application that uses a single MQTT connection for both publishing and subscribing to the topics **news** and **complaints**. The MQTT broker is running on the local computer, using the loopback address 127.0.0.1 (localhost).

```
dotnet new avalonia.app -n Mosquitto1
cd Mosquitto1
dotnet add package MQTTnet
```

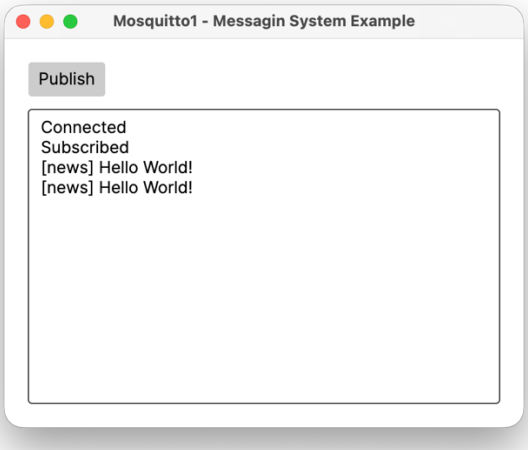


Figure 1 – Output of the Mosquitto1 project.

4.1.1. Implement the user interface using the following controls (`MainWindow.axaml`):

```
<Window xmlns="https://github.com/avaloniaui"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        xmlns:d="http://schemas.microsoft.com/expression/blend/2008"
        xmlns:mc="http://schemas.openxmlformats.org/markup-compatibility/2006"
        mc:Ignorable="d" d:DesignWidth="800" d:DesignHeight="450"
        x:Class="Mosquitto1.MainWindow"
        SizeToContent="WidthAndHeight"
        Title="Mosquitto1 - Messagin System Example">
    <StackPanel Margin="20" Spacing="10">
        <Button Name="btnPublish"
                Click="OnPublish_Click"
                Content="Publish"/>
        <ScrollViewer Height="250" Width="400">
            <TextBox Name="txtMessages"
                     IsReadOnly="True"
                     AcceptsReturn="True"
                     TextWrapping="Wrap"/>
        </ScrollViewer>
    </StackPanel>
</Window>
```

4.1.2. Implement a new class named **`MqttService`** to centralize all MQTT operations. The class should provide methods for connecting to the broker, publishing messages, subscribing to topics, unsubscribing from topics, and disconnecting when the application closes.

```
using System;
using System.Threading.Tasks;
using MQTTnet;

namespace Mosquitto1;

public class MqttService
{
    private readonly IMqttClient _client;

    public event Action<string, string>? MessageReceived; // (topic, payload)

    public MqttService()
    {
        var factory = new MqttClientFactory();
        _client = factory.CreateMqttClient();
        _client.ApplicationMessageReceivedAsync += e =>
        {
            string topic = e.ApplicationMessage.Topic;
            string payload = e.ApplicationMessage.ConvertPayloadToString();

            MessageReceived?.Invoke(topic, payload);
            return Task.CompletedTask;
        };
    }

    public async Task Connect()
    {
        var options = new MqttClientOptionsBuilder()
            .WithTcpServer("localhost", 1883) // host + port
            .Build();

        await _client.ConnectAsync(options);
    }

    public async Task Subscribe(string topic)
    {
        // await _client.SubscribeAsync(
        //     new MqttTopicFilterBuilder()
        //         .WithTopic(topic)
        //         .Build()
        // );
        await _client.SubscribeAsync(topic);
    }

    public async Task Publish(string topic, string message)
    {
        var mqttMessage = new MqttApplicationMessageBuilder()
        .WithTopic(topic)
        .WithPayload(message)
        .WithQualityOfServiceLevel(MQTTnet.Protocol.MqttQualityOfServiceLevel.AtLeastOnce)
        .Build();

        await _client.PublishAsync(mqttMessage);
    }

    public async Task Disconnect()
    {
        if (_client is not null)
            await _client.DisconnectAsync();
    }
}
```

4.1.3. Implement the **`MainWindow`** class so that all MQTT functionality is handled through the **`MqttService`** class.

```
using System;
using System.Collections.ObjectModel;
using System.Threading.Tasks;
using Avalonia.Controls;

using Avalonia.Threading;

namespace Mosquitto1;

public partial class MainWindow : Window
{
    private readonly MqttService _mqttService;

    public MainWindow()
    {
        InitializeComponent();

        _mqttService = new MqttService();
        _mqttService.MessageReceived += Mqtt_MessageReceived;

        Loaded += async (_, _) => await ConnetAndSubscribe();
    }

    public async Task ConnetAndSubscribe()
    {
        try
        {
            await _mqttService.Connect();
            txtMessages.Text += "Connected\n";
            await _mqttService.Subscribe("news");
            await _mqttService.Subscribe("complaints");
            txtMessages.Text += "Subscribed\n";
        }
        catch (System.Exception ex)
        {
            txtMessages.Text += $"Erro inesperado: {ex.Message}\n";
        }
    }

    private void Mqtt_MessageReceived(string topico, string payload)
    {
        Dispatcher.UIThread.Post(() =>
        {
            txtMessages.Text += $"[{topico}] {payload}\n";
        });
    }

    public async Task SendMessage()
    {
        try
        {
            await _mqttService.Publish("news", "Hello World!");
        }
        catch (System.Exception ex)
        {
            txtMessages.Text += $"Erro ao publicar: {ex.Message}\n";
        }
    }

    private async void OnPublish_Click(object? sender, Avalonia.Interactivity.RoutedEventArgs e)
    {
        try
        {
            await SendMessage();
```

```
        }
        catch (System.Exception ex)
        {
            OnStatusChanged($"Erro: {ex.Message}");
        }
    }
}
```

4.2. Exercise

Modify the previous implementation so that the available topics are displayed in the **ComboBox** when the form loads. The user should enter the message to be published in the **TextBox**. Additionally, add an "**Unsubscribe**" button that allows the user to unsubscribe from all currently subscribed topics.

Remember to implement the **MainWindow Closing** event to ensure that the MQTT connection is gracefully disconnected before the application exits.

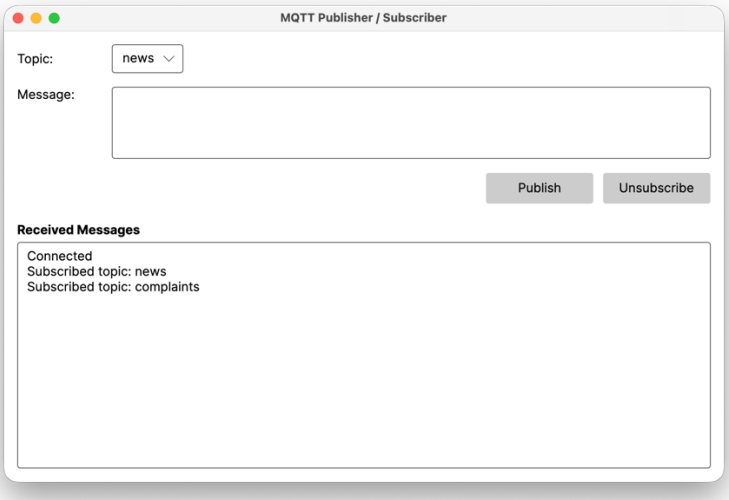


Figure 2 – Redesigned User Interface of the Mosquitto Application

.

4.3. Example 2 – Message broker on a Remote Computer

The following examples use **console** applications to exchange messages through a remote MQTT broker (e.g., 192.168.237.155, or the IP address of one of the computers in the laboratory). Test these examples using both a remote broker and the public MQTT broker available at **test.mosquitto.org**.

4.3.1. Create a separate console application for each example and verify its behaviour by exchanging messages between different clients connected to the same broker.

Subscriber:

```
using MQTTnet;

Console.WriteLine("Broker IP: ");
string? broker = Console.ReadLine();

Console.WriteLine("Topic:");
string? topic = Console.ReadLine();
```

```
var factory = new MqttClientFactory();
var client = factory.CreateMqttClient();

client.ApplicationMessageReceivedAsync += e =>
{
    string topic = e.ApplicationMessage.Topic;
    string payload = e.ApplicationMessage.ConvertPayloadToString();

    Console.WriteLine($"[{topic}] {payload}");
    return Task.CompletedTask;
};

var options = new MqttClientOptionsBuilder()
    .WithTcpServer(broker, 1883)
    .Build();

await client.ConnectAsync(options);

await client.SubscribeAsync(topic);

Console.WriteLine();
Console.WriteLine("Subscribed.");
Console.WriteLine("Waiting for messages...");
Console.WriteLine("Press ENTER to exit.");

Console.ReadLine();

if (client.IsConnected)
    await client.DisconnectAsync();
```

Publisher:

```
using MQTTnet;
Console.WriteLine("Broker IP: ");
string? broker = Console.ReadLine();

Console.WriteLine("Topic:");
string? topic = Console.ReadLine();

var factory = new MqttClientFactory();
var client = factory.CreateMqttClient();

var options = new MqttClientOptionsBuilder()
    .WithTcpServer(broker, 1883)
    .Build();

await client.ConnectAsync(options);

Console.WriteLine("Connected!");
Console.Write("Message (exit to quit): ");

string? message;
while ( (message = Console.ReadLine()) != null )
{
    if ( message.Equals("exit", StringComparison.OrdinalIgnoreCase))
        break;

    var mqttMessage = new MqttApplicationMessageBuilder()
    .WithTopic(topic)
    .WithPayload(message)
    .Build();

    await client.PublishAsync(mqttMessage);

    Console.WriteLine("Message sent!");
}
```

```
if (client.IsConnected)
    await client.DisconnectAsync();

Console.WriteLine("Disconnected.");
```

4.4. Example 3 – MQTT Chat Application

This example demonstrates a simple chat application named **MosquittoChatClient**, where each user can specify a nickname and a classroom. The application uses MQTT to exchange messages between connected users.

Error! Reference source not found. presents a screenshot of the application's graphical user interface (GUI). Study the implementation and test the application using both a local MQTT broker or a public broker (e.g.: test.mosquitto.org).

Download the **MosquittoChatClient** from the course web page (https://ead.ipleiria.pt/). Run the application and give it a try with your fellows.

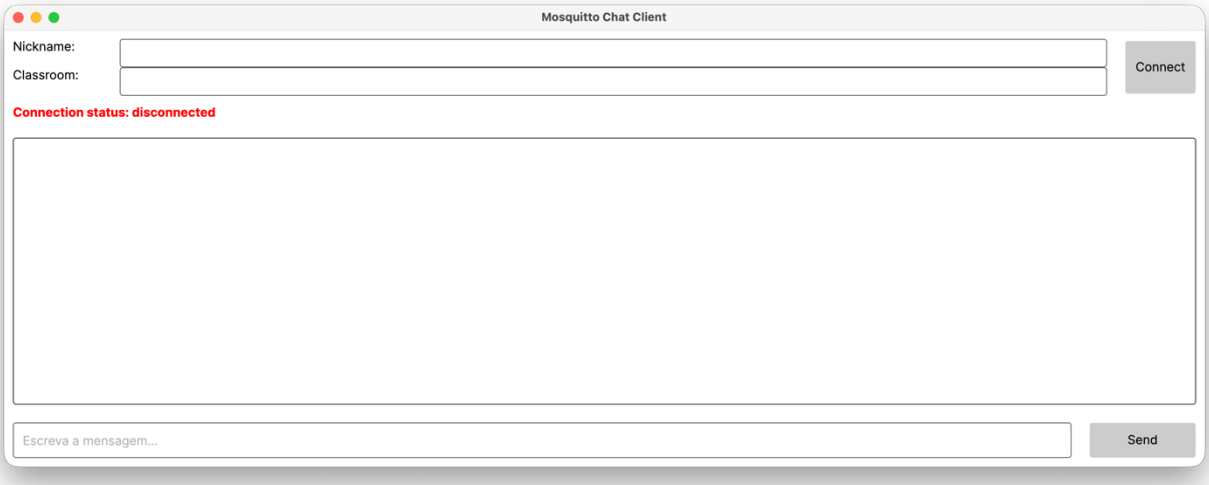


Figure 3 - MosquittoChatClient's GUI

4.5. Exercise - Improving the Chat Application

a) Although the *MosquittoChatClient* application works as expected, it has a major design weakness. Identify this weakness and discuss its potential consequences.

b) Fix the application's main weakness.

c) Implement a Web Service that returns the channel name to the MosquittoChatClient Application if the specified user is known. It is not mandatory to implement a system to register users (this can be made by hand). The user login and password are stored in an XML file.

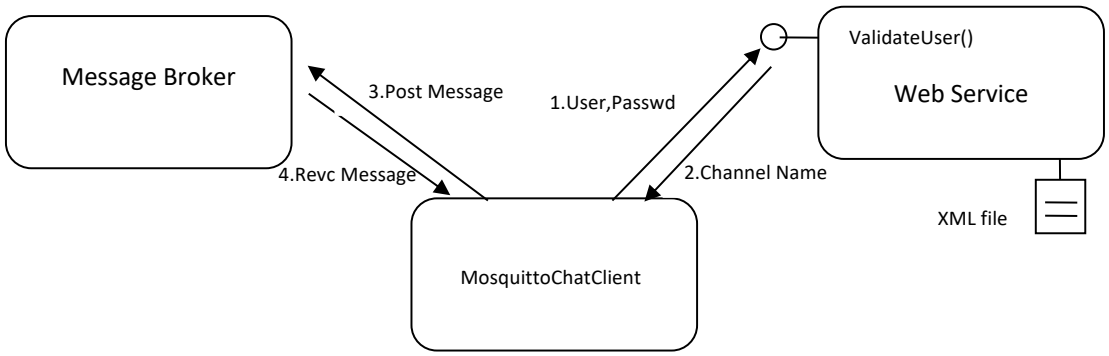


Figure 4 - High-level overview of the new MosquittoChatClient's architecture